An Explicit Counterexample to the Poisson Conjecture

Technical note, prepared and machine-verified by Claude (Anthropic)

July 20, 2026

Abstract

We exhibit an explicit polynomial endomorphism φ of the canonical Poisson algebra $P_3 = \mathbb{C}[x_1, x_2, x_3, p_1, p_2, p_3]$ that preserves the Poisson bracket but is not an automorphism. This refutes the Poisson Conjecture of Adjamagbo and van den Essen [4] for every $n \geq 3$. The construction applies the classical cotangent-lift reduction $\mathrm{PC}_n \Rightarrow \mathrm{JC}_n$, in contrapositive, to the non-injective Keller map $F : \mathbb{C}^3 \to \mathbb{C}^3$ with $\det J_F \equiv -2$ announced by Alpoge et al. on July 20, 2026 [8]. Every computational step is certified by two independent exact implementations (Appendix B); every remaining step is proved in full. In particular, the fiber $F^{-1}(-\frac{1}{4}, 0, 0)$ is determined exactly, by hand, and consists of precisely three points.

1 Statements and conventions

Throughout, $\mathbb{C}$ may be replaced by any field of characteristic zero; all arguments are purely formal (polynomial algebra, matrix identities over polynomial rings, and equality of mixed partial derivatives of polynomials, which holds monomial by monomial). No analysis is used.

Definition 1.1 (Jacobian Conjecture)**.** For $n \geq 1$, JC_n asserts: every polynomial map $F = (F_1, \ldots, F_n) : \mathbb{C}^n \to \mathbb{C}^n$ whose Jacobian determinant $\det J_F$, $(J_F)_{ij} = \partial F_i / \partial x_j$, is a nonzero constant is invertible with polynomial inverse [1, 2, 3].

Definition 1.2 (Canonical Poisson algebra)**.** $P_n = \mathbb{C}[x_1, \ldots, x_n, p_1, \ldots, p_n]$ equipped with the bracket

$$\{f, g\} \;=\; \sum_{k=1}^{n} \left(\frac{\partial f}{\partial x_k} \frac{\partial g}{\partial p_k} - \frac{\partial f}{\partial p_k} \frac{\partial g}{\partial x_k} \right),$$

so that $\{x_i, p_j\} = \delta_{ij}$ and $\{x_i, x_j\} = \{p_i, p_j\} = 0$. The bracket is $\mathbb{C}$-bilinear, antisymmetric, and a derivation in each argument. A *Poisson endomorphism* of P_n is a $\mathbb{C}$-algebra endomorphism φ with $\varphi(\{f, g\}) = \{\varphi(f), \varphi(g)\}$ for all $f, g \in P_n$; a *Poisson automorphism* is a bijective Poisson endomorphism. (The inverse of a bijective Poisson endomorphism is automatically Poisson: apply φ^{-1} to $\{\varphi\varphi^{-1}f, \varphi\varphi^{-1}g\} = \varphi\{\varphi^{-1}f, \varphi^{-1}g\}$.)

Definition 1.3 (Poisson Conjecture)**.** PC_n asserts: every Poisson endomorphism of P_n is an automorphism. The conjecture was introduced by Adjamagbo and van den Essen [4], who proved it equivalent (in the stable sense) to the Jacobian and Dixmier conjectures; see also [5, 6, 7].

Theorem 1.4 (Main theorem)**.** *Let $F = (F_1, F_2, F_3)$ be the polynomial map of Section 2, J_F its Jacobian matrix, and*

$$G \;=\; \big(J_F^T\big)^{-1} \;=\; -\tfrac{1}{2}\,\mathrm{adj}\big(J_F^T\big) \;\in\; M_3\big(\mathbb{C}[x_1, x_2, x_3]\big), \qquad Q_i \;=\; \sum_{k=1}^{3} G_{ik}\, p_k.$$

Then the $\mathbb{C}$-algebra endomorphism φ of P_3 determined by $\varphi(x_i) = F_i$, $\varphi(p_i) = Q_i$ is a Poisson endomorphism and is not an automorphism (indeed, not even an algebra automorphism). Consequently PC_n *is false for every $n \geq 3$.*

The fully expanded components of the associated polynomial map $\Phi = (F_1, F_2, F_3, Q_1, Q_2, Q_3) : \mathbb{C}^6 \to \mathbb{C}^6$ are printed in Appendix A.

2 The base map and its fiber

Write $(x, y, z) = (x_1, x_2, x_3)$ and set $u = 1 + xy$, so that $4 + 3xy = 3u + 1$. Define

$$\begin{aligned} F_1 &= u^3 z + y^2 u\,(3u+1), \\ F_2 &= y + 3xu^2 z + 3xy^2(3u+1), \\ F_3 &= 2x - 3x^2 y - x^3 z, \end{aligned}$$

of total degrees 7, 6, 4 respectively. This is the map announced in [8]; see Section 5 for provenance.

Lemma 2.1. $\det J_F \equiv -2$.

Proof. Each F_i is linear in z: writing $F = a(x,y)\,z + b(x,y)$ with

$$a = \begin{pmatrix} u^3 \\ 3xu^2 \\ -x^3 \end{pmatrix}, \qquad b = \begin{pmatrix} uy^2(3u+1) \\ y + 3xy^2(3u+1) \\ 2x - 3x^2 y \end{pmatrix},$$

the columns of J_F are $\partial F/\partial x = a_x z + b_x$, $\partial F/\partial y = a_y z + b_y$, and $\partial F/\partial z = a$. By multilinearity of the determinant in its columns,

$$\det J_F \;=\; c_2 z^2 + c_1 z + c_0, \qquad \begin{aligned} c_2 &= \det[\,a_x \mid a_y \mid a\,], \\ c_1 &= \det[\,a_x \mid b_y \mid a\,] + \det[\,b_x \mid a_y \mid a\,], \\ c_0 &= \det[\,b_x \mid b_y \mid a\,]. \end{aligned}$$

Here (using $u_x = y$, $u_y = x$)

$$a_x = \begin{pmatrix} 3u^2 y \\ 3u^2 + 6xuy \\ -3x^2 \end{pmatrix}, \quad a_y = \begin{pmatrix} 3u^2 x \\ 6x^2 u \\ 0 \end{pmatrix}, \quad b_x = \begin{pmatrix} y^3(6u+1) \\ 3y^2(3u+1) + 9xy^3 \\ 2 - 6xy \end{pmatrix}, \quad b_y = \begin{pmatrix} (3u+1)(xy^2 + 2uy) + 3xuy^2 \\ 1 + 6xy(3u+1) + 9x^2y^2 \\ -3x^2 \end{pmatrix}.$$

Vanishing of c_2 (by hand). Expanding $\det[\,a_x \mid a_y \mid a\,]$ along its third row $(-3x^2,\ 0,\ -x^3)$:

$$\begin{aligned} c_2 &= (-3x^2)\det\begin{pmatrix} 3u^2 x & u^3 \\ 6x^2 u & 3xu^2 \end{pmatrix} \;+\; (-x^3)\det\begin{pmatrix} 3u^2 y & 3u^2 x \\ 3u^2 + 6xuy & 6x^2 u \end{pmatrix} \\ &= (-3x^2)\big(9x^2u^4 - 6x^2u^4\big) \;+\; (-x^3)\big(18x^2u^3 y - 9xu^4 - 18x^2u^3 y\big) \\ &= -9x^4u^4 + 9x^4u^4 \;=\; 0. \end{aligned}$$

The identities $c_1 = 0$ and $c_0 = -2$ are elementary finite expansions of the two remaining 3×3 determinants displayed above; they are certified by exact expansion in two independent implementations (Appendix B, checks "c1" and "c0"), and the reader may reproduce each in about a page of routine algebra. Hence $\det J_F = c_0 = -2$. □

Remark 2.2. Two structural identities, verified by three lines of algebra each (substitute $xy = u - 1$), will pin down a fiber of F completely:

$$u\,F_2 - 3x\,F_1 = u\,y, \tag{A}$$
$$x^3F_1 + u^3F_3 = x\,u\,(u+1). \tag{B}$$

Indeed, $uF_2 - 3xF_1 = \big(uy + 3xu^3z + 9xu^2y^2 + 3xuy^2\big) - \big(3xu^3z + 9xu^2y^2 + 3xuy^2\big) = uy$; and $x^3F_1 + u^3F_3 = xu\big[3x^2uy^2 + x^2y^2 + 2u^2 - 3xyu^2\big] = xu\big[(u-1)^2(3u+1) + u^2(5-3u)\big] = xu(u+1)$.

Proposition 2.3 (Exact fiber)**.** *Let* $P_0 = (0, 0, -\frac{1}{4})$, $P_+ = (1, -\frac{3}{2}, \frac{13}{2})$, $P_- = (-1, \frac{3}{2}, \frac{13}{2})$. *Then*

$$F^{-1}\big(-\tfrac{1}{4},\,0,\,0\big) \;=\; \{P_0,\; P_+,\; P_-\},$$

three pairwise distinct points. In particular F *is not injective.*

Proof. The three points lie in the fiber. At P_0: $u = 1$, so $F = (z + 4y^2,\ y,\ 0)|_{(0,0,-1/4)} = (-\frac{1}{4}, 0, 0)$. At $P_\pm$: $xy = -\frac{3}{2}$, hence $u = -\frac{1}{2}$, $3u + 1 = -\frac{1}{2}$, $y^2 = \frac{9}{4}$, and

$$\begin{aligned}
F_1 &= \big(-\tfrac{1}{2}\big)^3\tfrac{13}{2} + \tfrac{9}{4}\big(-\tfrac{1}{2}\big)\big(-\tfrac{1}{2}\big) = -\tfrac{13}{16} + \tfrac{9}{16} = -\tfrac{1}{4},\\
F_2 &= y + 3x\cdot\tfrac{1}{4}\cdot\tfrac{13}{2} + 3x\cdot\tfrac{9}{4}\cdot\big(-\tfrac{1}{2}\big) = y + \tfrac{39}{8}x - \tfrac{27}{8}x = y + \tfrac{3}{2}x = 0,\\
F_3 &= 2x - 3x^2y - x^3z = x\big(2 - 3xy\big) - x^3z = x\cdot\tfrac{13}{2} - x\cdot\tfrac{13}{2} = 0,
\end{aligned}$$

using $x^2 = 1$ and $x^3 = x$ at $P_\pm$.

No other point lies in the fiber. Suppose $F(x, y, z) = (-\frac{1}{4}, 0, 0)$. Substituting into (A): $u\cdot 0 - 3x\big(-\frac{1}{4}\big) = uy$, i.e.

$$uy = \tfrac{3}{4}x. \tag{1}$$

Substituting into (B): $x^3\big(-\frac{1}{4}\big) + 0 = xu(u+1)$, i.e.

$$x\Big(u^2 + u + \tfrac{x^2}{4}\Big) = 0. \tag{2}$$

Case $x = 0$. Then $u = 1$ and $F(0, y, z) = (z + 4y^2,\ y,\ 0)$; the equations force $y = 0$, $z = -\frac{1}{4}$, i.e. the point P_0.

Case $x \neq 0$. Then $u^2 + u + \frac{x^2}{4} = 0$. Multiplying (1) by x and using $xy = u - 1$ gives $u(u-1) = \frac{3}{4}x^2$, i.e. $\frac{x^2}{4} = \frac{u^2-u}{3}$. Substituting,

$$u^2 + u + \frac{u^2 - u}{3} = 0 \quad\Longleftrightarrow\quad 4u^2 + 2u = 0 \quad\Longleftrightarrow\quad u \in \{0, -\tfrac{1}{2}\}.$$

If $u = 0$, (1) gives $x = 0$, a contradiction. Hence $u = -\frac{1}{2}$, so $\frac{x^2}{4} = \frac{1/4+1/2}{3} = \frac{1}{4}$, giving $x = \pm 1$ and $y = (u-1)/x = -\frac{3}{2x}$, i.e. $(x, y) = (1, -\frac{3}{2})$ or $(-1, \frac{3}{2})$. Finally $F_3 = 0$ determines $z = (2 - 3xy)/x^2 = (2 + \frac{9}{2})/1 = \frac{13}{2}$. Thus the point is P_+ or P_-.

The set-level conclusion is independently confirmed by a Gröbner-basis computation (Appendix B): the reduced lexicographic basis of the ideal $(4F_1 + 1,\ F_2,\ F_3)$ is $\big(3x + 2y,\ 12y^2 - 4z - 1,\ y(2z - 13),\ (2z - 13)(4z + 1)\big)$, whose zero set is exactly $\{P_0, P_+, P_-\}$. □

Corollary 2.4. *F is a Keller map (*$\det J_F \equiv -2 \in \mathbb{C}^\times$*) that is not injective, hence not invertible. Therefore* JC_3 *is false; padding with identity coordinates,* $\widehat{F}(x, w) = (F(x), w)$*, shows* JC_n *is false for every* $n \geq 3$*.*

Remark 2.5. If one insists on Jacobian determinant exactly 1, replace F by $(-\frac{1}{2}F_1, F_2, F_3)$: the determinant scales to 1 and the same three points collide (their images now agree at $(\frac{1}{8}, 0, 0)$).

3 The cotangent lift

The next three lemmas are stated for arbitrary n; only Lemma 2.1 and Proposition 2.3 are specific to our F. Lemma 3.2 is the classical reduction $\mathrm{PC}_n \Rightarrow \mathrm{JC}_n$ of [4] made explicit.

Lemma 3.1 (Generators suffice)**.** *Let $A = \mathbb{C}[w_1, \dots, w_m]$ carry a bracket $\{\cdot,\cdot\}$ that is $\mathbb{C}$-bilinear, antisymmetric, and a derivation in each argument, and let φ be a $\mathbb{C}$-algebra endomorphism of A with $\varphi(\{w_i, w_j\}) = \{\varphi(w_i), \varphi(w_j)\}$ for all i, j. Then $\varphi(\{f, g\}) = \{\varphi(f), \varphi(g)\}$ for all $f, g \in A$.*

Proof. Set $B_1(f,g) = \varphi(\{f,g\})$ and $B_2(f,g) = \{\varphi(f), \varphi(g)\}$. Both are $\mathbb{C}$-bilinear and satisfy the twisted Leibniz rule

$$B(fh, g) = \varphi(f)\, B(h,g) + \varphi(h)\, B(f,g), \qquad B(f, gh) = \varphi(g)\, B(f,h) + \varphi(h)\, B(f,g)\,:$$

for B_1 apply the Leibniz property of $\{\cdot,\cdot\}$ and then multiplicativity of φ; for B_2 apply multiplicativity of φ and then the Leibniz property. Taking $f = h = 1$ in the first rule gives $B(1,g) = 2B(1,g)$, so $B_i(1,g) = B_i(f,1) = 0$. For monomials f, g in the generators, induct on $\deg f + \deg g$: if $\deg f \geq 1$ write $f = w_i f'$ and use the first rule to reduce to smaller total degree; likewise in g; the base case $f = w_i$, $g = w_j$ holds by hypothesis. By bilinearity, $B_1 = B_2$ on all of A. □

Lemma 3.2 (Bracket relations)**.** *Let $F = (F_1, \dots, F_n) \in \mathbb{C}[x_1, \dots, x_n]^n$ with $\det J_F = c \in \mathbb{C}^\times$. Then $G := \big(J_F^T\big)^{-1} = c^{-1} \operatorname{adj}\big(J_F^T\big)$ has entries in $\mathbb{C}[x_1, \dots, x_n]$, and $Q_i := \sum_k G_{ik} p_k \in P_n$ satisfy, for all i, j:*

$$\textit{(i)}\ \{F_i, F_j\} = 0, \qquad \textit{(ii)}\ \{F_i, Q_j\} = \delta_{ij}, \qquad \textit{(iii)}\ \{Q_i, Q_j\} = 0.$$

Proof. Polynomiality of G is clear: the adjugate has polynomial entries and c is a nonzero scalar. The adjugate identities $M \operatorname{adj}(M) = \operatorname{adj}(M) M = \det(M)\, I$ over the commutative ring $\mathbb{C}[x_1, \dots, x_n]$ give the two-sided inverse relations $J_F^T G = G J_F^T = I$, and transposing, $J_F\, G^T = G^T J_F = I$.

(i) Both arguments lie in $\mathbb{C}[x_1, \dots, x_n]$, and every term of the bracket contains a derivative with respect to some p_k, which kills it.

(ii) Since $\partial F_i / \partial p_k = 0$ and $\partial Q_j / \partial p_k = G_{jk}$,

$$\{F_i, Q_j\} = \sum_k \frac{\partial F_i}{\partial x_k}\, G_{jk} = \sum_k (J_F)_{ik} \left(G^T\right)_{kj} = \left(J_F\, G^T\right)_{ij} = \delta_{ij}.$$

(iii) Write $M = J_F$, so $(M^T)_{ab} = \partial F_b / \partial x_a$ and $G = (M^T)^{-1}$. Since $\partial Q_i / \partial p_k = G_{ik}$ and $\partial Q_i / \partial x_k = \sum_l (\partial_k G_{il})\, p_l$,

$$\{Q_i, Q_j\} = \sum_k \Big(\frac{\partial Q_i}{\partial x_k} G_{jk} - G_{ik} \frac{\partial Q_j}{\partial x_k} \Big) = \sum_l p_l\, D_{ijl}, \qquad D_{ijl} := \sum_k \big(\partial_k G_{il}\, G_{jk} - G_{ik}\, \partial_k G_{jl} \big).$$

Differentiating $M^T G = I$ gives $\partial_k G = -G\, (\partial_k M^T)\, G$, where $(\partial_k M^T)_{ab} = \partial^2 F_b / \partial x_a \partial x_k =: H^b_{ak}$ is symmetric in (a,k) (equality of mixed partials of polynomials). Substituting,

$$D_{ijl} = -\sum_{a,k,b} H^b_{ak}\, G_{bl} \left(G_{ia} G_{jk} - G_{ik} G_{ja} \right).$$

The factor $G_{ia} G_{jk} - G_{ik} G_{ja}$ is antisymmetric under $a \leftrightarrow k$, while H^b_{ak} is symmetric; the contraction over (a,k) therefore vanishes, so $D_{ijl} = 0$ and $\{Q_i, Q_j\} = 0$. □

Lemma 3.3 (Point map)**.** *Let φ be a $\mathbb{C}$-algebra endomorphism of $\mathbb{C}[w_1, \dots, w_m]$ and let $\Phi = \big(\varphi(w_1), \dots, \varphi(w_m)\big) : \mathbb{C}^m \to \mathbb{C}^m$. Then $\varphi(f) = f \circ \Phi$ for every f. If φ is an automorphism, then Φ is bijective.*

Proof. The substitution map $f \mapsto f(\varphi(w_1), \dots, \varphi(w_m))$ is a $\mathbb{C}$-algebra endomorphism agreeing with φ on the generators, hence equal to it; evaluating at a point gives $\varphi(f) = f \circ \Phi$. If $\psi = \varphi^{-1}$ has point map Ψ, then for all f, $f \circ (\Psi \circ \Phi) = (\varphi \circ \psi)(f) = f$; taking $f = w_i$ yields $\Psi \circ \Phi = \mathrm{id}$, and symmetrically $\Phi \circ \Psi = \mathrm{id}$. Hence Φ is bijective. □

4 Proof of the main theorem

Proof of Theorem 1.4. By Lemma 2.1, $\det J_F = -2 \in \mathbb{C}^\times$, so Lemma 3.2 applies with $n = 3$, $c = -2$: the generators' brackets are preserved by φ ($\varphi(x_i) = F_i$, $\varphi(p_i) = Q_i$), and by Lemma 3.1, φ is a Poisson endomorphism of P_3.

The point map of φ is $\Phi(x, p) = \big(F(x),\, G(x)p\big)$. Since the momentum block is linear in p, $\Phi(P, 0) = (F(P), 0)$ for every $P \in \mathbb{C}^3$; by Proposition 2.3,

$$\Phi(P_0, 0) = \Phi(P_+, 0) = \Phi(P_-, 0) = \big((-\tfrac{1}{4}, 0, 0),\, 0\big),$$

with $(P_0, 0), (P_+, 0), (P_-, 0)$ pairwise distinct. So Φ is not injective, and by Lemma 3.3, φ is not an algebra automorphism; a fortiori not a Poisson automorphism. Hence PC_3 is false.

For $n > 3$, extend φ to φ_n on P_n by $\varphi_n(x_j) = x_j$, $\varphi_n(p_j) = p_j$ for $j > 3$. All generator brackets involving an index $j > 3$ are preserved: for $f \in \{F_i, Q_i : i \le 3\}$, which involve neither x_j nor p_j,

$$\{f, x_j\} = -\frac{\partial f}{\partial p_j} = 0, \qquad \{f, p_j\} = \frac{\partial f}{\partial x_j} = 0,$$

matching φ_n applied to $\{x_i, x_j\} = \{p_i, x_j\} = \{x_i, p_j\} = \{p_i, p_j\} = 0$ ($i \le 3 < j$), while the brackets among $\{x_j, p_j : j > 3\}$ are fixed verbatim. The point map of φ_n collides at the padded points $(P_\bullet, 0, \dots, 0)$, so φ_n is again a Poisson endomorphism and not an automorphism. Hence PC_n is false for all $n \ge 3$. □

Corollary 4.1. *Combining Corollary 2.4 with the classical implication $\mathrm{DC}_n \Rightarrow \mathrm{JC}_n$ [3, Thm. 10.4.2], the Dixmier conjecture fails for the Weyl algebra A_n for every $n \ge 3$.*

Remark 4.2. The block-triangular structure $J_\Phi = \begin{pmatrix} J_F & 0 \\ * & G \end{pmatrix}$ gives $\det J_\Phi = \det(J_F) \det(G) = c \cdot c^{-1} = 1$. Thus Φ is itself an explicit non-invertible polynomial map $\mathbb{C}^6 \to \mathbb{C}^6$ with Jacobian determinant identically 1 which moreover preserves the canonical symplectic form: the verified 6×6 identity $J_\Phi \, \Lambda \, J_\Phi^T = \Lambda$, with $\Lambda = \left(\begin{smallmatrix} 0 & I \\ -I & 0 \end{smallmatrix}\right)$, is exactly the statement that all generator brackets are preserved.

Remark 4.3. The cases $n \le 2$ of PC_n are untouched by this construction: $\mathrm{PC}_n \Rightarrow \mathrm{JC}_n$ and $\mathrm{JC}_{2n} \Rightarrow \mathrm{PC}_n$ [4], and JC_2 remains open, while JC_4 is now false, so neither implication settles PC_1 or PC_2.

5 Provenance and verification

Provenance. The map F was transcribed from the public announcement threads of July 20, 2026 [8], in which the counterexample is credited to L. Alpoge, a collaborator identified in the thread as

"Matthew," and the language model Claude Fable 5. At the time of writing, no refereed manuscript or preprint by the announcers could be located, and the encyclopedic record still lists the Jacobian conjecture as open pending review. Disclosure: this note was prepared by Claude (Anthropic), the same model family credited in the announcement. For that reason the note is written to be logically independent of the announcement: no property of F is used on anyone's authority. Everything required — Lemma 2.1 and Proposition 2.3 — is proved above, with the two residual polynomial expansions certified as described below. If the announcers' intended map differs from the transcription, the present results stand for the displayed F regardless.

Verification methodology. All polynomial identities asserted in this note were verified by exact computation in two independent implementations sharing no code: (1) the computer-algebra system SymPy (exact rational arithmetic, 17 checks), and (2) a self-contained integer-arithmetic multivariate polynomial engine written for this note, in which every identity is cleared of denominators and checked over $\mathbb{Z}$ (11 checks, including term-by-term agreement of the two constructions of F). A third layer evaluates $\det J_F$ at 25 random rational points through SymPy's numeric determinant path. A Gröbner-basis computation independently confirms Proposition 2.3. All 29 checks pass; the certification log and the complete verification program are reproduced in Appendix B. Verifying a polynomial identity by exact expansion over $\mathbb{Q}$ is a finite, deterministic computation; the two independent implementations exist to exclude implementation error, not mathematical doubt.

References

[1] O.-H. Keller, *Ganze Cremona-Transformationen*, Monatsh. Math. Phys. **47** (1939), 299–306.

[2] H. Bass, E. Connell, D. Wright, *The Jacobian conjecture: reduction of degree and formal expansion of the inverse*, Bull. Amer. Math. Soc. (N.S.) **7** (1982), 287–330.

[3] A. van den Essen, *Polynomial Automorphisms and the Jacobian Conjecture*, Progress in Mathematics 190, Birkhäuser, Basel, 2000.

[4] K. Adjamagbo, A. van den Essen, *A proof of the equivalence of the Dixmier, Jacobian and Poisson conjectures*, Acta Math. Vietnam. **32** (2007), no. 2–3, 205–214.

[5] K. Adjamagbo, A. van den Essen, *On the equivalence of the Jacobian, Dixmier and Poisson conjectures in any characteristic*, arXiv:math/0608009.

[6] Y. Tsuchimoto, *Endomorphisms of Weyl algebra and p-curvatures*, Osaka J. Math. **42** (2005), no. 2, 435–452.

[7] A. Belov-Kanel, M. Kontsevich, *The Jacobian conjecture is stably equivalent to the Dixmier conjecture*, Mosc. Math. J. **7** (2007), no. 2, 209–218.

[8] Public announcement threads, July 20, 2026: `https://x.com/jdlichtman/status/2079066717762863249`; `https://x.com/AndrewCurran_/status/2079081226217066891`; `https://news.ycombinator.com/item?id=48973869`.

A Fully expanded components of Φ

Below, $\Phi = (F_1, F_2, F_3, Q_1, Q_2, Q_3)$ with $Q_i = -\frac{1}{2}\sum_k \mathrm{adj}(J_F^T)_{ik}\, p_k$; term counts are 19, 24, 34 for Q_1, Q_2, Q_3, of total degrees 9, 10, 12. A machine-readable copy accompanies this note (`phi_expanded.txt`).

$$
\begin{aligned}
F_1 = {} & x^3y^3z + 3x^2y^4 + 3x^2y^2z \\
& + 7xy^3 + 3xyz + 4y^2 \\
& + z
\end{aligned}
$$

$$
\begin{aligned}
F_2 = {} & 3x^3y^2z + 9x^2y^3 + 6x^2yz \\
& + 12xy^2 + 3xz + y
\end{aligned}
$$

$$
F_3 = -x^3z - 3x^2y + 2x
$$

$$
\begin{aligned}
Q_1 = {} & 3p_1x^6yz - 9p_3x^5yz^2 \\
& + 9p_1x^5y^2 - 45p_3x^4y^2z \\
& + 3p_1x^5z + 3p_2x^4yz \\
& - 9p_3x^4z^2 - 54p_3x^3y^3 \\
& + 3p_1x^4y + 9p_2x^3y^2 \\
& - 30p_3x^3yz + 3p_2x^3z \\
& - 27p_3x^2y^2 - 4p_1x^3 \\
& + 3p_2x^2y + 9p_3x^2z \\
& + 21p_3xy - 3p_2x \\
& + p_3
\end{aligned}
$$

$$
\begin{aligned}
Q_2 = {} & -\frac{3p_1x^6y^2z}{2} + \frac{9p_3x^5y^2z^2}{2} \\
& - \frac{9p_1x^5y^3}{2} + \frac{45p_3x^4y^3z}{2} \\
& - 3p_1x^5yz - \frac{3p_2x^4y^2z}{2} \\
& + 9p_3x^4yz^2 + 27p_3x^3y^4 \\
& - 6p_1x^4y^2 - \frac{9p_2x^3y^3}{2} \\
& + \frac{75p_3x^3y^2z}{2} - \frac{3p_1x^4z}{2} \\
& - 3p_2x^3yz + \frac{9p_3x^3z^2}{2} \\
& + \frac{81p_3x^2y^3}{2} + \frac{p_1x^3y}{2} \\
& - 6p_2x^2y^2 + \frac{21p_3x^2yz}{2} \\
& - \frac{3p_2x^2z}{2} + 3p_3xy^2 \\
& + \frac{3p_1x^2}{2} - 3p_3xz \\
& - 8p_3y + p_2
\end{aligned}
$$

$$
\begin{aligned}
Q_3 = & -\frac{3p_1x^6y^4z}{2} + \frac{9p_3x^5y^4z^2}{2} \\
& - \frac{9p_1x^5y^5}{2} + \frac{45p_3x^4y^5z}{2} \\
& - 6p_1x^5y^3z - \frac{3p_2x^4y^4z}{2} \\
& + 18p_3x^4y^3z^2 + 27p_3x^3y^6 \\
& - 15p_1x^4y^4 - \frac{9p_2x^3y^5}{2} \\
& + \frac{165p_3x^3y^4z}{2} - 9p_1x^4y^2z \\
& - 6p_2x^3y^3z + 27p_3x^3y^2z^2 \\
& + \frac{189p_3x^2y^5}{2} - 16p_1x^3y^3 \\
& - 15p_2x^2y^4 + 108p_3x^2y^3z \\
& - 6p_1x^3yz - 9p_2x^2y^2z \\
& + 18p_3x^2yz^2 + 111p_3xy^4 \\
& - \frac{9p_1x^2y^2}{2} - \frac{33p_2xy^3}{2} \\
& + \frac{117p_3xy^2z}{2} - \frac{3p_1x^2z}{2} \\
& - 6p_2xyz + \frac{9p_3xz^2}{2} \\
& + \frac{89p_3y^3}{2} + \frac{3p_1xy}{2} \\
& - 6p_2y^2 + \frac{21p_3yz}{2} \\
& - \frac{3p_2z}{2} + \frac{p_1}{2}
\end{aligned}
$$

B Machine certification

Certification log

```
======================================================================
LAYER 1: sympy exact symbolic verification
======================================================================
PASS  det JF == -2 identically (sympy)
PASS  F(P0) = F(P+) = F(P-) = (-1/4, 0, 0) (sympy)
PASS  P0, P+, P- pairwise distinct
PASS  identity (A): u*F2 - 3x*F1 == u*y
PASS  identity (B): x^3*F1 + u^3*F3 == x*u^2 + x*u
PASS  F = a(x,y)*z + b(x,y) with b independent of z
PASS  c2 = det[a_x | a_y | a] == 0
PASS  c1 = det[a_x | b_y | a] + det[b_x | a_y | a] == 0
PASS  c0 = det[b_x | b_y | a] == -2
PASS  (JF^T)*G == I
PASS  G polynomial (no variables in denominators)
PASS  {F_i, F_j} == 0 for all i,j (sympy)
PASS  {F_i, Q_j} == delta_ij for all i,j (sympy)
PASS  {Q_i, Q_j} == 0 for all i,j (sympy)
```

```
PASS  JPhi * Lambda * JPhi^T == Lambda (full 6x6 identity)
PASS  dF/dp block is zero (JPhi block-triangular)
PASS  det JPhi == 1 (= det JF * det G)

======================================================================
LAYER 2: independent exact integer-arithmetic engine
======================================================================
PASS  engine F1 matches sympy F1 term-for-term
PASS  engine F2 matches sympy F2 term-for-term
PASS  engine F3 matches sympy F3 term-for-term
PASS  det JF == -2 identically (engine)
PASS  (JF^T) * adj(JF^T) == -2 I (engine)
PASS  adj(JF^T) * (JF^T) == -2 I (engine)
PASS  {F_i, F_j} == 0                (engine, cleared)
PASS  {F_i, Qt_j} == -2 delta_ij     (engine, cleared; Qt = -2Q)
PASS  {Qt_i, Qt_j} == 0              (engine, cleared)
PASS  F(P0) = F(P+) = F(P-) = (-1/4, 0, 0) (engine)
PASS  Phi(P0,0) = Phi(P+,0) = Phi(P-,0) = ((-1/4,0,0), 0)

======================================================================
LAYER 3: numeric spot checks of det JF at random rational points
======================================================================
PASS  det JF == -2 at 25 random rational points (numeric det path)

======================================================================
RESULT: ALL CHECKS PASSED
======================================================================
```

The Gröbner-basis cross-check of Proposition 2.3 (run separately) returned the reduced lexicographic basis $\big(3x+2y,\ 12y^2-4z-1,\ y(2z-13),\ (2z-13)(4z+1)\big)$ and the exact solution set $\{P_0, P_+, P_-\}$.

Verification program

```
#!/usr/bin/env python3
"""
Verification suite for a counterexample to the Poisson Conjecture PC_3.

Layer 1: sympy exact symbolic computation.
Layer 2: independent exact integer-arithmetic polynomial engine (no sympy
         code paths shared; all identities cleared of denominators).
Layer 3: numeric spot checks of det JF at random rational points.

Outputs:
  /home/claude/verify_output.txt   -- certification log
  /home/claude/phi_expanded.txt    -- fully expanded components of Phi
  /home/claude/appendixA.tex       -- LaTeX for expanded components
"""
import sys, random
from fractions import Fraction

LOG = []
def log(s=""):
    print(s)
    LOG.append(str(s))

FAILED = []
def check(name, ok):
    log(("PASS  " if ok else "FAIL  ") + name)
```

```
    if not ok:
        FAILED.append(name)

# ======================================================================
# LAYER 1: SYMPY
# ======================================================================
import sympy as sp

x, y, z, p1, p2, p3 = sp.symbols('x y z p1 p2 p3')
XV = [x, y, z]; PV = [p1, p2, p3]; ALLV = XV + PV
u = 1 + x*y

F1 = u**3*z + y**2*u*(4 + 3*x*y)
F2 = y + 3*x*u**2*z + 3*x*y**2*(4 + 3*x*y)
F3 = 2*x - 3*x**2*y - x**3*z
F  = sp.Matrix([F1, F2, F3])

log("="*70); log("LAYER 1: sympy exact symbolic verification"); log("="*70)

J = F.jacobian(sp.Matrix(XV))
detJ = sp.expand(J.det())
check("det JF == -2 identically (sympy)", detJ == -2)

# --- collisions ---------------------------------------------------
P0 = (sp.Integer(0), sp.Integer(0), sp.Rational(-1,4))
Pp = (sp.Integer(1),  sp.Rational(-3,2), sp.Rational(13,2))
Pm = (sp.Integer(-1), sp.Rational(3,2),  sp.Rational(13,2))
TARGET = (sp.Rational(-1,4), sp.Integer(0), sp.Integer(0))
pts = [P0, Pp, Pm]
imgs = [tuple(sp.nsimplify(v) for v in F.subs(dict(zip(XV, pt)))) for pt in pts]
check("F(P0) = F(P+) = F(P-) = (-1/4, 0, 0) (sympy)",
      all(tuple(sp.simplify(a - b) for a, b in zip(im, TARGET)) == (0,0,0) for im in imgs))
check("P0, P+, P- pairwise distinct", len(set(pts)) == 3)

# --- structural identities (A), (B) -----------------------------------
idA = sp.expand(u*F2 - 3*x*F1 - u*y)
idB = sp.expand(x**3*F1 + u**3*F3 - x*u**2 - x*u)
check("identity (A): u*F2 - 3x*F1 == u*y", idA == 0)
check("identity (B): x^3*F1 + u^3*F3 == x*u^2 + x*u", idB == 0)

# --- z-decomposition of det JF: det = c2 z^2 + c1 z + c0 -------------
a_vec = sp.Matrix([sp.diff(Fi, z) for Fi in [F1, F2, F3]])       # coefficient of z
b_vec = sp.Matrix([sp.expand(Fi - a_vec[i]*z) for i, Fi in enumerate([F1, F2, F3])])
check("F = a(x,y)*z + b(x,y) with b independent of z",
      all(sp.diff(b_vec[i], z) == 0 for i in range(3)))
ax = a_vec.diff(x); ay = a_vec.diff(y)
bx = b_vec.diff(x); by = b_vec.diff(y)
def col_det(c1_, c2_, c3_):
    return sp.expand(sp.Matrix.hstack(c1_, c2_, c3_).det())
c2 = col_det(ax, ay, a_vec)
c1 = sp.expand(col_det(ax, by, a_vec) + col_det(bx, ay, a_vec))
c0 = col_det(bx, by, a_vec)
check("c2 = det[a_x | a_y | a] == 0", c2 == 0)
check("c1 = det[a_x | b_y | a] + det[b_x | a_y | a] == 0", c1 == 0)
check("c0 = det[b_x | b_y | a] == -2", c0 == -2)

# --- cotangent lift ---------------------------------------------------
G = sp.expand((J.T).adjugate() / detJ)          # (JF^T)^{-1}, entries in Q[x,y,z]
check("(JF^T)*G == I", sp.simplify(J.T*G - sp.eye(3)) == sp.zeros(3,3))
```

```
check("G polynomial (no variables in denominators)",
      all(not sp.denom(sp.together(e)).free_symbols for e in G))
Pcol = sp.Matrix(PV)
Q = sp.expand(G*Pcol)

def PB(f, g):
    return sp.expand(sum(sp.diff(f, XV[k])*sp.diff(g, PV[k])
                         - sp.diff(f, PV[k])*sp.diff(g, XV[k]) for k in range(3)))

ok_xx = all(PB(F[i], F[j]) == 0 for i in range(3) for j in range(3))
ok_xp = all(sp.expand(PB(F[i], Q[j]) - (1 if i == j else 0)) == 0
            for i in range(3) for j in range(3))
ok_pp = all(PB(Q[i], Q[j]) == 0 for i in range(3) for j in range(3))
check("{F_i, F_j} == 0 for all i,j (sympy)", ok_xx)
check("{F_i, Q_j} == delta_ij for all i,j (sympy)", ok_xp)
check("{Q_i, Q_j} == 0 for all i,j (sympy)", ok_pp)

# --- full 6x6 symplectic-matrix identity ------------------------------
Phi = sp.Matrix([F1, F2, F3, Q[0], Q[1], Q[2]])
JPhi = Phi.jacobian(sp.Matrix(ALLV))
Lam = sp.Matrix(sp.BlockMatrix([[sp.zeros(3,3), sp.eye(3)],
                                [-sp.eye(3),   sp.zeros(3,3)]]))
symp = sp.expand(JPhi*Lam*JPhi.T - Lam)
check("JPhi * Lambda * JPhi^T == Lambda (full 6x6 identity)", symp == sp.zeros(6,6))
check("dF/dp block is zero (JPhi block-triangular)",
      all(sp.diff(F[i], PV[j]) == 0 for i in range(3) for j in range(3)))
check("det JPhi == 1 (= det JF * det G)", sp.expand(detJ*G.det()) == 1)

# ======================================================================
# LAYER 2: INDEPENDENT INTEGER POLYNOMIAL ENGINE (no sympy)
# ======================================================================
log(""); log("="*70)
log("LAYER 2: independent exact integer-arithmetic engine")
log("="*70)

NV = 6   # exponent order: x, y, z, p1, p2, p3

def pconst(c): return {(0,)*NV: c} if c else {}
def pvar(i):
    e = [0]*NV; e[i] = 1
    return {tuple(e): 1}
def padd(a, b):
    r = dict(a)
    for m, c in b.items():
        s = r.get(m, 0) + c
        if s: r[m] = s
        elif m in r: del r[m]
    return r
def psub(a, b): return padd(a, {m: -c for m, c in b.items()})
def pmul(a, b):
    r = {}
    for m1, c1 in a.items():
        for m2, c2 in b.items():
            m = tuple(i + j for i, j in zip(m1, m2))
            s = r.get(m, 0) + c1*c2
            if s: r[m] = s
            elif m in r: del r[m]
    return r
def pscale(a, k): return {m: c*k for m, c in a.items()} if k else {}
```

```
def pdiff(a, i):
    r = {}
    for m, c in a.items():
        if m[i]:
            mm = list(m); mm[i] -= 1
            mm = tuple(mm)
            s = r.get(mm, 0) + c*m[i]
            if s: r[mm] = s
            elif mm in r: del r[mm]
    return r
def peval(a, vals):
    s = Fraction(0)
    for m, c in a.items():
        t = Fraction(c)
        for i, e in enumerate(m):
            if e: t *= vals[i]**e
        s += t
    return s

ex, ey, ez = pvar(0), pvar(1), pvar(2)
ep = [pvar(3), pvar(4), pvar(5)]
eu = padd(pconst(1), pmul(ex, ey))
c4_3xy = padd(pconst(4), pscale(pmul(ex, ey), 3))

eF1 = padd(pmul(pmul(pmul(eu, eu), eu), ez),
           pmul(pmul(pmul(ey, ey), eu), c4_3xy))
eF2 = padd(padd(ey, pscale(pmul(pmul(ex, pmul(eu, eu)), ez), 3)),
           pscale(pmul(pmul(ex, pmul(ey, ey)), c4_3xy), 3))
eF3 = padd(psub(pscale(ex, 2), pscale(pmul(pmul(ex, ex), ey), 3)),
           {(3,0,1,0,0,0): -1})
eF = [eF1, eF2, eF3]

# cross-check the two constructions define the same polynomials
for i, Fi in enumerate([F1, F2, F3]):
    Psy = sp.Poly(sp.expand(Fi), *ALLV)
    dsy = {tuple(mon): int(coef) for mon, coef in Psy.terms()}
    check(f"engine F{i+1} matches sympy F{i+1} term-for-term", dsy == eF[i])

eJ = [[pdiff(eF[i], j) for j in range(3)] for i in range(3)]

def det2(m00, m01, m10, m11): return psub(pmul(m00, m11), pmul(m01, m10))
def det3(M):
    return padd(psub(pmul(M[0][0], det2(M[1][1], M[1][2], M[2][1], M[2][2])),
                     pmul(M[0][1], det2(M[1][0], M[1][2], M[2][0], M[2][2]))),
                pmul(M[0][2], det2(M[1][0], M[1][1], M[2][0], M[2][1])))

edet = det3(eJ)
check("det JF == -2 identically (engine)", edet == pconst(-2))

def transpose(M): return [[M[j][i] for j in range(3)] for i in range(3)]
def adj3(M):
    # adj(M)[i][j] = (-1)^(i+j) * det(minor of M deleting row j, col i)
    A = [[None]*3 for _ in range(3)]
    for i in range(3):
        for j in range(3):
            rows = [r for r in range(3) if r != j]
            cols = [c for c in range(3) if c != i]
            mn = det2(M[rows[0]][cols[0]], M[rows[0]][cols[1]],
                      M[rows[1]][cols[0]], M[rows[1]][cols[1]])
```

```
            A[i][j] = pscale(mn, (-1)**(i + j))
    return A
def matmul(A, B):
    return [[padd(padd(pmul(A[i][0], B[0][j]), pmul(A[i][1], B[1][j])),
                  pmul(A[i][2], B[2][j])) for j in range(3)] for i in range(3)]

eJT = transpose(eJ)
eAdjT = adj3(eJT)                       # = -2 * G, integer entries
prod1 = matmul(eJT, eAdjT)
prod2 = matmul(eAdjT, eJT)
ok1 = all(prod1[i][j] == (pconst(-2) if i == j else {}) for i in range(3) for j in range(3))
ok2 = all(prod2[i][j] == (pconst(-2) if i == j else {}) for i in range(3) for j in range(3))
check("(JF^T) * adj(JF^T) == -2 I (engine)", ok1)
check("adj(JF^T) * (JF^T) == -2 I (engine)", ok2)

# Qt_i := -2 * Q_i  (integer-cleared momenta)
eQt = [padd(padd(pmul(eAdjT[i][0], ep[0]), pmul(eAdjT[i][1], ep[1])),
            pmul(eAdjT[i][2], ep[2])) for i in range(3)]

def ePB(f, g):
    r = {}
    for k in range(3):
        r = padd(r, psub(pmul(pdiff(f, k), pdiff(g, k + 3)),
                         pmul(pdiff(f, k + 3), pdiff(g, k))))
    return r

ok_xx = all(ePB(eF[i], eF[j]) == {} for i in range(3) for j in range(3))
ok_xp = all(ePB(eF[i], eQt[j]) == (pconst(-2) if i == j else {})
            for i in range(3) for j in range(3))
ok_pp = all(ePB(eQt[i], eQt[j]) == {} for i in range(3) for j in range(3))
check("{F_i, F_j} == 0                (engine, cleared)", ok_xx)
check("{F_i, Qt_j} == -2 delta_ij     (engine, cleared; Qt = -2Q)", ok_xp)
check("{Qt_i, Qt_j} == 0              (engine, cleared)", ok_pp)

# collisions in the engine
epts = [(Fraction(0), Fraction(0), Fraction(-1, 4)),
        (Fraction(1), Fraction(-3, 2), Fraction(13, 2)),
        (Fraction(-1), Fraction(3, 2), Fraction(13, 2))]
etarget = (Fraction(-1, 4), Fraction(0), Fraction(0))
ok_col = all(tuple(peval(eF[i], list(pt) + [Fraction(0)]*3) for i in range(3)) == etarget
             for pt in epts)
check("F(P0) = F(P+) = F(P-) = (-1/4, 0, 0) (engine)", ok_col)

# lifted collisions: Phi(P,0) has momentum block 0 automatically (linear in p)
ok_lift = all(tuple(peval(q, list(pt) + [Fraction(0)]*3) for q in eQt) == (0, 0, 0)
              for pt in epts)
check("Phi(P0,0) = Phi(P+,0) = Phi(P-,0) = ((-1/4,0,0), 0)", ok_lift)

# ======================================================================
# LAYER 3: numeric spot checks (independent code path for det)
# ======================================================================
log(""); log("="*70)
log("LAYER 3: numeric spot checks of det JF at random rational points")
log("="*70)
random.seed(20260720)
ok_num = True
for _ in range(25):
    vals = {x: sp.Rational(random.randint(-40, 40), random.randint(1, 9)),
            y: sp.Rational(random.randint(-40, 40), random.randint(1, 9)),
```

```
            z: sp.Rational(random.randint(-40, 40), random.randint(1, 9))}
    Jn = J.subs(vals)                       # numeric 3x3, numeric det path
    if Jn.det() != -2:
        ok_num = False; break
check("det JF == -2 at 25 random rational points (numeric det path)", ok_num)

# ======================================================================
# EXPORTS
# ======================================================================
Qs = [sp.expand(Q[i]) for i in range(3)]
with open("/home/claude/phi_expanded.txt", "w") as f:
    f.write("Fully expanded components of Phi : C^6 -> C^6\n")
    f.write("Phi(x,y,z,p1,p2,p3) = (F1,F2,F3,Q1,Q2,Q3),  Q = (JF^T)^{-1} p\n")
    f.write("Convention: {f,g} = sum_k (df/dx_k dg/dp_k - df/dp_k dg/dx_k)\n")
    f.write("="*70 + "\n\n")
    for i, e in enumerate([F1, F2, F3]):
        f.write(f"F{i+1} = {sp.expand(e)}\n\n")
    for i, e in enumerate(Qs):
        f.write(f"Q{i+1} = {e}\n\n")
    f.write("Integer-cleared momenta (Qt_i = -2*Q_i):\n\n")
    for i, e in enumerate(Qs):
        f.write(f"Qt{i+1} = {sp.expand(-2*e)}\n\n")
    f.write("Term counts / total degrees:\n")
    for nm, e in [("F1",F1),("F2",F2),("F3",F3),("Q1",Qs[0]),("Q2",Qs[1]),("Q3",Qs[2])]:
        ee = sp.expand(e)
        f.write(f"  {nm}: {len(sp.Add.make_args(ee))} terms, total degree {sp.total_degree(ee)}\n")

# LaTeX appendix: chunked expanded polynomials
def poly_lines(expr, name, per_line=3):
    P = sp.Poly(sp.expand(expr), *ALLV)
    terms = sorted(P.terms(), key=lambda t: (-sum(t[0]), tuple(-e for e in t[0])))
    parts = []
    for mono, coef in terms:
        m = sp.S.One
        for v, e in zip(ALLV, mono):
            m *= v**e
        parts.append(coef*m)
    lines, cur = [], []
    for t in parts:
        cur.append(t)
        if len(cur) == per_line:
            lines.append(cur); cur = []
    if cur: lines.append(cur)
    out = []
    for k, chunk in enumerate(lines):
        s = ""
        for j, t in enumerate(chunk):
            ts = sp.latex(t)
            if j == 0 and k == 0:
                s += ts
            else:
                s += (" " + ts) if ts.startswith("-") else (" + " + ts)
        out.append(s)
    body = ("\\\\\n&\\quad ").join(out)
    return f"\\begin{{align*}}\n{name} ={{}}& {body}\n\\end{{align*}}\n"

with open("/home/claude/appendixA.tex", "w") as f:
    f.write("% auto-generated: fully expanded components of Phi\n")
    for i, e in enumerate([F1, F2, F3]):
```

```
        f.write(poly_lines(e, f"F_{i+1}", per_line=3))
    for i, e in enumerate(Qs):
        f.write(poly_lines(e, f"Q_{i+1}", per_line=2))

log(""); log("="*70)
if FAILED:
    log(f"RESULT: {len(FAILED)} CHECK(S) FAILED: {FAILED}")
else:
    log("RESULT: ALL CHECKS PASSED")
log("="*70)

with open("/home/claude/verify_output.txt", "w") as f:
    f.write("\n".join(LOG) + "\n")

sys.exit(1 if FAILED else 0)
```